

03_Agent实战项目1

导入任何一个你可以使用的模型

In [1]:

```
from langchain_deepseek import ChatDeepSeek

model = ChatDeepSeek(model="deepseek-chat",
                      api_key = '',
                      base_url = 'https://api.deepseek.com')
```

None of PyTorch, TensorFlow >= 2.0, or Flax have been found. Models won't be available and only tokenizers, configuration and file/data utilities can be used.

导入数据

In [2]:

```
import numpy as np
import pandas as pd

telco = pd.read_csv('telco_data.csv')
pd.set_option('max_colwidth',50)

telco.head(5)
```

Out[2]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multipl
0	7590-VHVEG	Female	0	Yes	No	1	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	No
4	9237-HQITU	Female	0	No	No	2	Yes	

5 rows × 21 columns

导入工具并验证

```
In [3]: # LangChain 内置代码解释器工具功能
from langchain_experimental.tools import PythonAstREPLTool

ast = PythonAstREPLTool(locals={"telco": telco}) #把已有的dataframe导入工具上下文
ast_ans = ast.invoke("telco['SeniorCitizen'].mean()") #给定代码，让工具执行
```

C:\Users\yanchen\AppData\Local\Temp\ipykernel_32060\2562297597.py:2: DeprecationWarning: `langchain-experimental` is being sunset and is no longer actively maintained. See <https://github.com/langchain-ai/langchain-experimental/issues/87> for details.

```
from langchain_experimental.tools import PythonAstREPLTool
```

```
In [4]: # 手工验证是否正确
res = telco['SeniorCitizen'].mean()

print(f'Agent 计算结果: {ast_ans}')
print(f'实际结果: {res}')
```

Agent 计算结果: 0.1621468124378816
实际结果: 0.1621468124378816

编写对查询生成代码公式的 Agent

```
In [5]: # 定义第一个 Agent 的输出格式

from pydantic import BaseModel, Field

class write_formula_format(BaseModel):
    """对用户的数据查询需求编写对应公式"""
    search_formula: str = Field(..., description="对用户的数据查询需求编写对应公式，只
```

```
In [6]: # 编写对查询生成代码公式的 Agent

from langchain.agents import create_agent
from langchain.tools import tool

@tool
def write_formula(query):
    """
    对用户的需求 query，编写对应查询计算的代码公式
    :param query: str，必填 - 用户输入的查询需求
    :return: search_formula: str，对用户的数据查询需求编写对应公式，只允许使用 pandas 和
    """

    system_prompt = f"""
    你是一位专业的数据分析师，你会耐心思考然后解答用户提出的问题：
    1. 你可以访问一个名为 `telco` 的 pandas 数据表，你可以使用telco.head().to_markdown()
    2. 请根据用户提出的问题，编写 Python 代码来回答。记住只返回代码，不返回其他内容，而且
    """

    conversation = [
        {"role": "system", "content": '''
        1. 你可以访问一个名为 `telco` 的 pandas 数据表，你可以使用telco.head().to_markdown()
        2. 首先，请根据用户提出的问题，编写 Python 代码来回答。记住只返回代码，不返回其他
        3. 仅返回计算需要的代码公式'''},
```

```

        {"role": "user", "content": query},
    ]

    f_agent = create_agent(model = model,
                           tools = [ast],
                           system_prompt = system_prompt,
                           response_format = write_formula_format
                           )

    formula_res = f_agent.invoke({"messages": conversation})

    search_formula = formula_res['structured_response'].search_formula

    return search_formula

```

编写 Agent 提示词

```

In [7]: system_prompt = f"""
你是一位专业的数据分析师，你会耐心思考然后解答用户提出的问题：
1. 你可以访问一个名为 `telco` 的 pandas 数据表，你可以使用telco.head().to_markdown() 查
2. 首先，请根据用户提出的问题，编写 Python 代码来回答。记住只返回代码，不返回其他内容，而且
3. 将生成的代码送入 ast (PythonAstREPLTool) 工具进行运行解读，最后返回结果。
"""

```

定义输出格式，创建Agent

```

In [8]: class res_format(BaseModel):
        """对用户的query进行回答，分别为用户问题，计算公式与计算结果"""
        query: str = Field(..., description="对用户的 query 进行精炼总结")
        formula: str = Field(..., description="计算查询的 python 代码公式")
        result: str = Field(..., description="根据formula计算的结果")

        agent = create_agent(model = model,
                              tools = [ast, write_formula],
                              system_prompt = system_prompt,
                              response_format = res_format
                              )

```

对 df 表格进行数据分析

```

In [9]: response = agent.invoke(
        {"messages": [{"role": "user", "content": "我有一张表，名为 telco，请帮我计算 Mont
        )

```

```
<class 'pandas.core.frame.DataFrame'>
```

```

In [10]: print(response['structured_response'].query)
print(response['structured_response'].formula)
print(response['structured_response'].result)

```

计算telco表中MonthlyCharges字段的均值

```
telco['MonthlyCharges'].mean()
```

64.76169246059918

```
In [11]: for i in response['messages']:  
         print(i.__class__)
```

```
<class 'langchain_core.messages.human.HumanMessage'>  
<class 'langchain_core.messages.ai.AIMessage'>  
<class 'langchain_core.messages.tool.ToolMessage'>  
<class 'langchain_core.messages.ai.AIMessage'>  
<class 'langchain_core.messages.tool.ToolMessage'>  
<class 'langchain_core.messages.ai.AIMessage'>  
<class 'langchain_core.messages.tool.ToolMessage'>  
<class 'langchain_core.messages.ai.AIMessage'>  
<class 'langchain_core.messages.tool.ToolMessage'>  
<class 'langchain_core.messages.ai.AIMessage'>  
<class 'langchain_core.messages.tool.ToolMessage'>
```

```
In [12]: for i in response['messages']:  
         print(i, end = '\n\n')
```

```
content='我有一张表, 名为 telco, 请帮我计算 MonthlyCharges 字段的均值。' additional_kwa
rgs={} response_metadata={} id='fd9aea14-8ab3-42fa-9e11-604448603e62'
```

```
content='' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'c
ompletion_tokens': 65, 'prompt_tokens': 676, 'total_tokens': 741, 'completion_tokens
_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens':
0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 676}, 'model_provide
r': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d0
2d0_prod0820_fp8_kvcache_20260402', 'id': 'ecb4bb10-a8b8-4b39-b6f9-7bd2b269ea45', 'f
inish_reason': 'tool_calls', 'logprobs': None} id='lc_run--019ed503-28d1-71e3-a50b-4
77740704cf6-0' tool_calls=[{'name': 'python_repl_ast', 'args': {'query': 'import pan
das as pd\n# 先查看telco数据表的基本信息\nprint(telco.head().to_markdown())\n'}, 'id':
'call_00_7YIqFnVnLebPrSw5u2W31106', 'type': 'tool_call'}] invalid_tool_calls=[] usag
e_metadata={'input_tokens': 676, 'output_tokens': 65, 'total_tokens': 741, 'input_to
ken_details': {'cache_read': 0}, 'output_token_details': {}}
```

```
content="ImportError: Missing optional dependency 'tabulate'. Use pip or conda to i
ninstall tabulate." name='python_repl_ast' id='f11333e5-792f-48f9-89f6-771ae1dc0be2' t
ool_call_id='call_00_7YIqFnVnLebPrSw5u2W31106'
```

```
content='' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'c
ompletion_tokens': 64, 'prompt_tokens': 780, 'total_tokens': 844, 'completion_tokens
_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 64
0}, 'prompt_cache_hit_tokens': 640, 'prompt_cache_miss_tokens': 140}, 'model_provide
r': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d0
2d0_prod0820_fp8_kvcache_20260402', 'id': '5dc1551d-7b00-469d-9195-4fbb9f02e623', 'f
inish_reason': 'tool_calls', 'logprobs': None} id='lc_run--019ed503-30d6-7f23-bd0b-1
4360fdc89a8-0' tool_calls=[{'name': 'python_repl_ast', 'args': {'query': '# 检查telc
o是否存在, 并查看前几行数据\nprint(type(telco))\nprint(telco.head())\n'}, 'id': 'call_
00_97ASuODfJ7MePZ5EFT668157', 'type': 'tool_call'}] invalid_tool_calls=[] usage_meta
data={'input_tokens': 780, 'output_tokens': 64, 'total_tokens': 844, 'input_token_de
tails': {'cache_read': 640}, 'output_token_details': {}}
```

```
content='  customerID  gender  SeniorCitizen  Partner  Dependents  tenure  PhoneServic
e  \\\n0  7590-VHVEG  Female  0  Yes  No  1  No
\n1  5575-GNVDE  Male  0  No  No  34  Yes  \n2
3668-QPYBK  Male  0  No  No  2  Yes  \n3  779
5-CFOCW  Male  0  No  No  45  No  \n4  9237-H
QITU  Female  0  No  No  2  Yes  \n\n  Mult
ipleLines  InternetService  OnlineSecurity  ...  DeviceProtection  \\\n0  No  phone serv
ice  DSL  No  ...  No  \n1  No
DSL  Yes  ...  Yes  \n2  No  DSL
Yes  ...  No  \n3  No  phone service  DSL  Yes
...  Yes  \n4  No  Fiber optic  No  ...
No  \n\n  TechSupport  StreamingTV  StreamingMovies  Contract  PaperlessBilling
\\\n0  No  No  No  Month-to-month  Yes  \n1
No  No  No  One year  No  \n2  No
No  No  Month-to-month  Yes  \n3  Yes  No
No  One year  No  \n4  No  No  No  M
onth-to-month  Yes  \n\n  PaymentMethod  MonthlyCharges  To
talCharges  Churn  \n0  Electronic check  29.85  29.85  No
\n1  Mailed check  56.95  1889.5  No  \n2
Mailed check  53.85  108.15  Yes  \n3  Bank transfer (automatic)
42.30  1840.75  No  \n4  Electronic check  70.70  151.
65  Yes  \n\n[5 rows x 21 columns]\n' name='python_repl_ast' id='6bd9e31c-d773-4c80
-ae9d-b391c049f008' tool_call_id='call_00_97ASuODfJ7MePZ5EFT668157'
```

```
content='' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 47, 'prompt_tokens': 1252, 'total_tokens': 1299, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 768}}, 'prompt_cache_hit_tokens': 768, 'prompt_cache_miss_tokens': 484}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '5657f7be-c6d7-45fa-a874-bd33e1a31aca', 'finish_reason': 'tool_calls', 'logprobs': None} id='lc_run--019ed503-36df-7c00-994a-33251bb603e4-0' tool_calls=[{'name': 'write_formula', 'args': {'query': '计算telco表中MonthlyCharges字段的均值'}}, {'id': 'call_00_i8r6MscYzII02py0sSkq6753', 'type': 'tool_call'}] invalid_tool_calls=[] usage_metadata={'input_tokens': 1252, 'output_tokens': 47, 'total_tokens': 1299, 'input_token_details': {'cache_read': 768}, 'output_token_details': {}}
```

```
content="telco['MonthlyCharges'].mean()" name='write_formula' id='fa152c91-c0f2-4ed2-a284-0457341691eb' tool_call_id='call_00_i8r6MscYzII02py0sSkq6753'
```

```
content='' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 54, 'prompt_tokens': 1328, 'total_tokens': 1382, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 1280}}, 'prompt_cache_hit_tokens': 1280, 'prompt_cache_miss_tokens': 48}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '04a330a3-5042-4cc5-86fa-5ff1359e6c64', 'finish_reason': 'tool_calls', 'logprobs': None} id='lc_run--019ed503-450f-76b3-a7e6-b7bbb442c051-0' tool_calls=[{'name': 'python_repl_ast', 'args': {'query': "mean_value = telco['MonthlyCharges'].mean()\nmean_value\n"}, {'id': 'call_00_H44Rfb3ULnca6mTVo5uw9821', 'type': 'tool_call'}] invalid_tool_calls=[] usage_metadata={'input_tokens': 1328, 'output_tokens': 54, 'total_tokens': 1382, 'input_token_details': {'cache_read': 1280}, 'output_token_details': {}}
```

```
content='64.76169246059918' name='python_repl_ast' id='f0eba409-d034-4b9e-9b79-80bfc538a0f0' tool_call_id='call_00_H44Rfb3ULnca6mTVo5uw9821'
```

```
content='' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 94, 'prompt_tokens': 1408, 'total_tokens': 1502, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 1280}}, 'prompt_cache_hit_tokens': 1280, 'prompt_cache_miss_tokens': 128}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'cf2440c8-167e-4733-ae1c-52043b36c3e3', 'finish_reason': 'tool_calls', 'logprobs': None} id='lc_run--019ed503-4a41-79d1-8301-04a460485686-0' tool_calls=[{'name': 'res_format', 'args': {'query': '计算telco表中MonthlyCharges字段的均值', 'formula': "telco['MonthlyCharges'].mean()", 'result': '64.76169246059918'}, {'id': 'call_00_l7nSPbvy1hndpsp05r4B7160', 'type': 'tool_call'}] invalid_tool_calls=[] usage_metadata={'input_tokens': 1408, 'output_tokens': 94, 'total_tokens': 1502, 'input_token_details': {'cache_read': 1280}, 'output_token_details': {}}
```

```
content='Returning structured response: query=\'计算telco表中MonthlyCharges字段的均值\' formula="telco[\'MonthlyCharges\'].mean()" result=\'64.76169246059918\' name=\'res_format\' id=\'649e6a67-eda0-4c08-a761-61b3ce2b4c24\' tool_call_id=\'call_00_l7nSPbvy1hndpsp05r4B7160\''
```

In [13]: response

```

Out[13]: {'messages': [HumanMessage(content='我有一张表, 名为 telco, 请帮我计算 MonthlyCharges 字段的均值.', additional_kwargs={}, response_metadata={}, id='fd9aea14-8ab3-42fa-9e11-604448603e62'),
  AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'tokens_usage': {'completion_tokens': 65, 'prompt_tokens': 676, 'total_tokens': 741, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 676}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'ecb4bb10-a8b8-4b39-b6f9-7bd2b269ea45', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019ed503-28d1-71e3-a50b-477740704cf6-0', tool_calls=[{'name': 'python_repl_ast', 'args': {'query': 'import pandas as pd\n# 先查看telco数据表的基本信息\nprint(telco.head().to_markdown())\n'}, 'id': 'call_00_7YIqFnVnLebPrSw5u2W31106', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 676, 'output_tokens': 65, 'total_tokens': 741, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}),
  ToolMessage(content="ImportError: Missing optional dependency 'tabulate'. Use pip or conda to install tabulate.", name='python_repl_ast', id='f11333e5-792f-48f9-89f6-771ae1dc0be2', tool_call_id='call_00_7YIqFnVnLebPrSw5u2W31106'),
  AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'tokens_usage': {'completion_tokens': 64, 'prompt_tokens': 780, 'total_tokens': 844, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 640}, 'prompt_cache_hit_tokens': 640, 'prompt_cache_miss_tokens': 140}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '5dc1551d-7b00-469d-9195-4fbb9f02e623', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019ed503-30d6-7f23-bd0b-14360fdc89a8-0', tool_calls=[{'name': 'python_repl_ast', 'args': {'query': '# 检查telco是否存在, 并查看前几行数据\nprint(type(telco))\nprint(telco.head())\n'}, 'id': 'call_00_97ASuODfJ7MePZ5EFT668157', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 780, 'output_tokens': 64, 'total_tokens': 844, 'input_token_details': {'cache_read': 640}, 'output_token_details': {}}),
  ToolMessage(content='
  customerID  gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \\\n0  7590-VHVEG  Female  0  Yes  No  1
1  No  \n1  5575-GNVDE  Male  0  No  No  34
Yes  \n2  3668-QPYBK  Male  0  No  No  2  Y
es  \n3  7795-CFOCW  Male  0  No  No  45  N
o  \n4  9237-HQITU  Female  0  No  No  2  Yes
\n\n  MultipleLines  InternetService  OnlineSecurity  ...  DeviceProtection  \\\n0  No phone service  DSL  No  ...  No  \n1
No  DSL  Yes  ...  Yes  \n2  No
DSL  Yes  ...  No  \n3  No phone service  DSL
Yes  ...  Yes  \n4  No  Fiber optic  No
...  No  \n\n  TechSupport  StreamingTV  StreamingMovies  Contra
ct  PaperlessBilling  \\\n0  No  No  No  Month-to-month
Yes  \n1  No  No  No  One year  No
\n2  No  No  No  Month-to-month  Yes  \n3
Yes  No  No  One year  No  \n4  No
No  No  Month-to-month  Yes  \n\n  PaymentMe
thod  MonthlyCharges  TotalCharges  Churn  \n0  Electronic check  2
9.85  29.85  No  \n1  Mailed check  56.95  188
9.5  No  \n2  Mailed check  53.85  108.15  Yes  \n3
Bank transfer (automatic)  42.30  1840.75  No  \n4  Electr
onic check  70.70  151.65  Yes  \n\n[5 rows x 21 columns]\n', name
='python_repl_ast', id='6bd9e31c-d773-4c80-ae9d-b391c049f008', tool_call_id='call_00_97ASuODfJ7MePZ5EFT668157'),

```

```

AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'tokens_usage': {'completion_tokens': 47, 'prompt_tokens': 1252, 'total_tokens': 1299, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 768}, 'prompt_cache_hit_tokens': 768, 'prompt_cache_miss_tokens': 484}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '5657f7be-c6d7-45fa-a874-bd33e1a31aca', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019ed503-36df-7c00-994a-33251bb603e4-0', tool_calls=[{'name': 'write_formula', 'args': {'query': '计算telco表中MonthlyCharges字段的均值'}, 'id': 'call_00_i8r6MscYzII02py0sSkq6753', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 1252, 'output_tokens': 47, 'total_tokens': 1299, 'input_token_details': {'cache_read': 768}, 'output_token_details': {}}),

```

```

ToolMessage(content="telco['MonthlyCharges'].mean()", name='write_formula', id='fa152c91-c0f2-4ed2-a284-0457341691eb', tool_call_id='call_00_i8r6MscYzII02py0sSkq6753'),

```

```

AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'tokens_usage': {'completion_tokens': 54, 'prompt_tokens': 1328, 'total_tokens': 1382, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 1280}, 'prompt_cache_hit_tokens': 1280, 'prompt_cache_miss_tokens': 48}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '04a330a3-5042-4cc5-86fa-5ff1359e6c64', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019ed503-450f-76b3-a7e6-b7bbb442c051-0', tool_calls=[{'name': 'python_repl_ast', 'args': {'query': "mean_value = telco['MonthlyCharges'].mean()\nmean_value\n"}, 'id': 'call_00_H44Rfb3ULnca6mTVo5uw9821', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 1328, 'output_tokens': 54, 'total_tokens': 1382, 'input_token_details': {'cache_read': 1280}, 'output_token_details': {}}),

```

```

ToolMessage(content='64.76169246059918', name='python_repl_ast', id='f0eba409-d034-4b9e-9b79-80bfc538a0f0', tool_call_id='call_00_H44Rfb3ULnca6mTVo5uw9821'),

```

```

AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'tokens_usage': {'completion_tokens': 94, 'prompt_tokens': 1408, 'total_tokens': 1502, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 1280}, 'prompt_cache_hit_tokens': 1280, 'prompt_cache_miss_tokens': 128}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'cf2440c8-167e-4733-ae1c-52043b36c3e3', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019ed503-4a41-79d1-8301-04a460485686-0', tool_calls=[{'name': 'res_format', 'args': {'query': '计算telco表中MonthlyCharges字段的均值', 'formula': "telco['MonthlyCharges'].mean()", 'result': '64.76169246059918'}, 'id': 'call_00_17nSPbvy1hndpsp05r4B7160', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 1408, 'output_tokens': 94, 'total_tokens': 1502, 'input_token_details': {'cache_read': 1280}, 'output_token_details': {}}),

```

```

ToolMessage(content='Returning structured response: query=\'计算telco表中MonthlyCharges字段的均值\' formula="telco[\'MonthlyCharges\'].mean()" result=\'64.76169246059918\'', name='res_format', id='649e6a67-eda0-4c08-a761-61b3ce2b4c24', tool_call_id='call_00_17nSPbvy1hndpsp05r4B7160']),

```

```

'structured_response': res_format(query='计算telco表中MonthlyCharges字段的均值', formula="telco['MonthlyCharges'].mean()", result='64.76169246059918'))

```